

Day 1–27 Revision Sheet

Linux · Shell Scripting · Git & GitHub · Task 2 & Task 3

How to use this sheet: read the **Concept** box first to refresh the idea, then cover the **Quiz Recap** and try answering from memory before checking. Revisit this every few days — spaced repetition is what actually moves things into long-term memory.

LINUX

1. Navigating the File System, Create/Move/Delete

Concept

Absolute path	Starts with / — always means the same place, from root. e.g. /var/log
Relative path	From your current directory. e.g. cd logs only works if 'logs' is right here
mkdir -p a/b/c	Creates nested directories in one shot; no error if they already exist
mv src dest	Both move AND rename — there is no separate rename command
rm -rf	Recursive + Force delete — no trash bin, no undo. Use with care
cd -	Jump back to the previous directory you were in

Quiz Recap

Q1. Difference between `cd /logs` and `cd logs`?

Understood after correction — Absolute = full address from root, always the same place. Relative = 'next door', depends on where you're standing.

Q2. Create `projects/2026/devops` in one command?

Correct — `mkdir -p projects/2026/devops`

Q3. Why is `rm -rf` dangerous?

Correct (refined) — `-r` = recursive (deletes whole tree, not just empty folders), `-f` = force (no confirmation, no errors). Combined = silent, total deletion.

Q4. Jump to `/var/log` then instantly back?

Corrected — `cd /var/log` then `cd -` (no need to retype the original path)

2. Managing Processes — List, Kill, Background/Foreground

Concept

ps aux / ps -ef	Snapshot of all processes right now — good for scripting/grep
top / htop	Live, refreshing view — good for active diagnosis
kill PID	Sends SIGTERM (15) — polite request, process can clean up first
kill -9 PID	Sends SIGKILL (9) — kernel force-terminates, no cleanup, last resort
Ctrl+Z / bg / fg	Suspend foreground job, then resume it in background / foreground
nohup cmd &	Keeps running after logout by ignoring the SIGHUP signal

Quiz Recap

Q1. kill PID vs kill -9 PID — why does it matter?

Correct (refined) — SIGTERM lets the process catch the signal and clean up (close files, save state). SIGKILL bypasses the process entirely — kernel force-kills it, risking corruption/locked resources. Always try plain kill first.

Q2. Forgot & on a long script, need terminal back?

Correct — Ctrl+Z to suspend, then bg to resume it running in the background.

Q3. Why does nohup cmd & survive terminal close but cmd & doesn't?

Correct — Closing a terminal sends SIGHUP to all its jobs, killing them. nohup makes the process ignore SIGHUP.

Q4. ps aux vs top — when to use which?

Recap (was skipped) — ps aux = one-time snapshot, good for grep/scripts. top = live, interactive, sorts by CPU/mem, good for active 'what's eating my CPU' diagnosis.

3. systemd — Start, Stop, Enable, Status

Concept

start / stop / restart	Controls the service right now, this session only
enable / disable	Controls auto-start on next boot — independent of start/stop
reload	Re-reads config without dropping active connections (if supported)
systemctl status svc	Shows Active state, whether it's enabled, and recent log lines
journalctl -u svc -xe	Scoped, detailed logs for one service — best first move after a failure
PID 1	systemd is the first process at boot; if it dies, kernel panics

Quiz Recap

Q1. Service runs now but won't survive tonight's reboot — fix?

Correct — It's not enabled. Run systemctl enable svc (or enable --now).

Q2. restart vs reload, why prefer reload in production?

Correct — restart drops active connections; reload re-reads config live without interrupting in-flight requests. Not all services support reload.

Q3. systemctl status shows Active: failed — next move?

Correct (scope it to the service) — journalctl -u -xe — scoped to that service, avoids noise from the whole system log.

Q4. What is PID 1 and why is systemd that PID?

Correct — First process started at boot. If PID 1 dies, the kernel panics — nothing above it to restart things. It's also the reparent target for orphaned processes.

4. vi/vim and nano — Reading & Editing Files

Concept

nano	WYSIWYG editor, shortcuts shown on-screen (Ctrl+O save, Ctrl+X exit)
vim modes	Normal mode = commands. Insert mode (i/a/o) = actual typing. Command mode (:) = save/quit/search
Escape hatch	Esc then :q! — quit vim without saving, works from any mess
dd / yy / p	Cut line / copy line / paste — normal mode commands, not just deletion
:w vs :wq	:w saves only (checkpoint while still editing). :wq saves and exits

Quiz Recap

Q1. Why does vim even have modes?

Learned (was unsure) — vi was built for old terminals with limited keys — every key needed to double as an editing command. Modes separate 'keyboard as command panel' (Normal) from 'keyboard as typing' (Insert), enabling powerful multi-key commands like dd, yy.

Q2. Exit vim discarding all changes?

Correct — Esc, then :q!

Q3. :w vs :wq — when use :w alone?

Correct — :w saves a checkpoint mid-edit (e.g. before a risky change) without leaving vim. :wq saves and quits.

Q4. What does dd do differently from Backspace?

Correct (add: it's a cut, not just delete) — dd deletes the WHOLE current line in one command AND copies it to the buffer (cut), so it can be pasted with p. Backspace only removes one character while actively typing.

5. Troubleshooting CPU / Memory / Disk — top, free, df, du

Concept

free -h	Watch 'available', not 'free' — Linux uses spare RAM for reclaimable disk cache
df -h	Disk usage per mounted filesystem/partition
du -sh path/*	Disk usage per directory — drills into what's taking the space
df vs du mismatch	A deleted file still held open by a process keeps its disk blocks allocated — df sees them, du (walks the directory tree) can't
Finding the culprit	lsuf grep deleted — shows PID holding a deleted-but-open file

Quiz Recap

Q1. free -h shows only 500Mi free — worried?

Correct (terminology: free itself is unused RAM, separate from buff/cache) — Not necessarily. Free RAM (not cache) — check 'available' (free + reclaimable cache), that's the real usable number. Low free + high available = normal.

Q2. Explain the deleted-file-still-using-space scenario.

Correct (refined mechanism) — rm removes the filename from the directory (so du can't see it), but if a process still holds the file open, the kernel keeps the disk blocks allocated until every handle closes — df counts those blocks. Find it with lsuf | grep deleted.

Q3. df vs du — one sentence each.

Correct (sharpened) — df reports space at the filesystem/partition level (raw block usage). du reports space at the file/directory level by walking the tree.

Q4. Find 5 largest files under /var/log.

Correct — `du -ah /var/log | sort -rh | head -5`

6. Linux File System Hierarchy

Concept

<code>/</code>	Root — top of the ONE tree everything hangs off (no drive letters)
<code>/etc</code>	System-wide config files (nginx.conf, ssh config, passwd)
<code>/var</code>	Variable/growing data — logs (<code>/var/log</code>), databases, caches
<code>/home</code> vs <code>/root</code>	<code>/home/user</code> = regular user homes. <code>/root</code> = home of the root user only (NOT the same as <code>/</code>)
<code>/tmp</code>	Temporary files, wiped on reboot — never store anything important here
<code>/proc</code>	Virtual filesystem — live kernel/process data, not real files on disk

Quiz Recap

Q1. A service crashed — which directory first, and why?

Corrected (mixed up logs vs config order) — Logs first: `/var/log` or `journalctl -u` . Logs tell you WHAT happened; only after that do you check `/etc` config to see WHY it's set up that way.

Q2. Where should a one-off temporary script live?

Correct — `/tmp` — cleared on reboot, appropriate for throwaway files.

Q3. Difference between `/` and `/root`?

Correct — `/` is the top of the entire filesystem tree. `/root` is just one user's (the root user's) home directory — same category as `/home/username`.

Q4. What does 'virtual filesystem' mean for `/proc`?

Correct — Not real data on disk — generated live by the kernel on read. e.g. `cat /proc/uptime` gives a different number every time, unlike a static config file.

7. Users, Groups, Passwords

Concept

<code>useradd -m</code>	<code>-m</code> creates the home directory too (often forgotten)
<code>usermod -aG group user</code>	<code>-a</code> = APPEND to group. Omitting <code>-a</code> REPLACES all existing group memberships!
<code>/etc/passwd</code>	Account metadata only (username, UID, home, shell) — world-readable, no passwords
<code>/etc/shadow</code>	Actual password hashes — root-readable only, split out for security
<code>id</code> vs <code>groups</code>	<code>id</code> = full detail (UID, GID, all groups with numbers). <code>groups</code> = just group names

Quiz Recap

Q1. Risk of `usermod -G` vs `usermod -aG`?

Correct (mechanics right, wording tightened) — `-G` (no `-a`) REPLACES all of a user's group memberships with just the one given — silently removing them from `sudo/docker/etc`. `-aG` appends without disturbing existing groups.

Q2. Why doesn't `/etc/passwd` store passwords anymore?

Learned (was unsure of the 'why') — It must be world-readable for tools to look up account info, which would expose password hashes to any user. Hashes were split into `/etc/shadow`, readable only by root.

Q3. `useradd bob` without `-m` — what breaks?

Correct — No home directory — no `.bashrc/.profile/SSH` keys, broken or incomplete login environment.

Q4. `id` vs `groups` — when to use which?

Correct — groups = quick list of group names. id = full picture: UID, primary GID, and all supplementary groups with numeric IDs — use when you need exact IDs for permission debugging.

8. File Permissions — chmod (Numeric & Symbolic)

Concept

<code>r=4, w=2, x=1</code>	Add per category: owner / group / others. e.g. 7=rwx, 5=r-x, 4=r--
<code>chmod 755</code>	Sets ALL THREE categories explicitly, overwriting whatever was there before
<code>chmod +x</code>	Only ADDS execute on top of existing r/w bits — does not touch or reset them
<code>chmod u+x / g-w / o=r</code>	Symbolic: surgical, incremental change to one category without affecting others
<code>755 vs +x</code>	755 = fixed exact state. +x = additive change to current state — NOT the same thing

Quiz Recap

Q1. Numeric equivalent of `-rw-r--r--` and what each can do?

Correct — 644 — owner: read+write, group: read-only, others: read-only.

Q2. chmod so ONLY owner has full access, nobody else anything?

Correct — `chmod 700`

Q3. File is 640, run `chmod +x` — new permissions?

Corrected — answer was incomplete — 640 (rw-r-----) becomes 750 (rwxr-x---). +x adds execute to owner and group (both had permissions already); others stays --- since it had nothing.

Q4. When use `chmod g+w` instead of numeric?

Correct — When you want to tweak ONE bit (e.g. add group write) without needing to know or overwrite the other existing permissions. Numeric requires specifying the full state; symbolic is surgical.

9. File Ownership — chown & chgrp

Concept

<code>chown user:group file</code>	Sets owner and group in one command
<code>chown -R user:group dir/</code>	Recursive — whole directory tree
<code>chown :group file</code>	Change ONLY the group (note the leading colon) — same effect as <code>chgrp</code>
Only root can chown	Changing OWNER needs elevated privileges — prevents giving away files / trust abuse
Permissions need correct ownership	<code>chmod</code> bits mean nothing if the owner/group are wrong — 'others' gets nothing regardless of generous bits

Quiz Recap

Q1. App deployed as 'deploy' user, needs 'www-data' access — fix ownership recursively?

Corrected — answered `chmod` instead of `chown` — `chown -R www-data:www-data /path/to/app`

Q2. `chown :devs file.txt` vs `chgrp devs file.txt`?

Correct — Functionally equivalent — `chgrp` is a dedicated shortcut for the group-only change.

Q3. Why can't regular users `chown` their own files to someone else?

Correct (concrete reasoning added) — It would let you give away files / dodge quotas / trick a root-owned process into trusting a file you planted. Ownership transfer is a trust boundary, so only root can do it.

Q4. `chmod 750` but wrong owner — does that fix access for the person who needs it?

Correct — No — `chmod`'s owner/group/others buckets only mean something once `chown/chgrp` correctly assign who IS the owner/group. Wrong owner = falls into 'others' bucket = gets nothing.

10. LVM — Logical Volume Manager

Concept

PV (Physical Volume)	A disk/partition initialized for LVM via pvcreate
VG (Volume Group)	Pool combining one or more PVs — disk boundaries stop mattering
LV (Logical Volume)	A 'virtual partition' carved from the VG pool — this is what you format & mount
Resize live	lvextend -L +10G then resize2fs — grow storage without unmounting
Combining disks	A VG spanning 2 disks lets you create an LV LARGER than any single disk

Quiz Recap

Q1. Explain PV -> VG -> LV in your own words.

Correct (PV needed pvcreate clarification) — PV = a disk prepared for LVM use (via pvcreate). Multiple PVs combine into a VG (a pooled capacity). The VG is then carved into LVs of any size needed.

Q2. LV running low, VG has free space — grow it?

Correct — lvextend -L +10G /dev/vg/lv, then resize2fs /dev/vg/lv

Q3. Real advantage of LVM over raw partitions?

Correct — Can grow storage live, in seconds, without unmounting — vs. backup/repartition/restore with raw partitions.

Q4. vgcreate myvg /dev/sdb /dev/sdc — what does combining disks let you do?

Corrected — answer went off-topic — Create a single logical volume LARGER than either physical disk alone — e.g. two 50GB disks pooled let you make a 90GB LV spanning both, transparently, as one volume to the filesystem.

11. Network Connectivity — ping, curl, netstat/ss, dig/nslookup

Concept

ping	Tests basic reachability (ICMP) — network layer only, not a specific service
curl	Tests actual HTTP(S) response — the application layer, what you usually really care about
ss -tulnp	Shows listening ports + which process owns them (t=TCP, u=UDP, l=listening, n=numeric, p=process)
dig / nslookup	DNS lookup — does the domain resolve to an IP at all?
Troubleshoot order	DNS -> ping -> ss (on server) -> curl. Check the LOWEST layer that could be broken first

Quiz Recap

Q1. Ping succeeds but website is down — next step?

Correct — curl the site to check if the application layer is actually responding.

Q2. Difference between what ping and curl test?

Correct — ping = basic network-layer reachability (is the host up). curl = application-layer response (is the actual service answering). Host can ping fine while its web server is down.

Q3. How to check if anything listens on port 443, and what do the flags mean?

Correct — ss -tulnp | grep :443 — t=TCP, u=UDP, l=listening only, n=numeric (skip DNS lookup, faster), p=show owning process.

Q4. Troubleshooting order for a website that won't load?

Corrected — DNS was put last instead of first — Correct order: dig/nslookup FIRST (does the domain even resolve?) -> ping (is that IP reachable?) -> ss -tulnp on the server (is something listening?) -> curl (is the app

responding correctly?). Check DNS first — if it fails, nothing downstream matters yet.

12. DNS, IP Addressing, Subnets, Common Ports

Concept

DNS resolution chain	Local cache -> resolver -> root servers -> TLD servers (.com) -> authoritative nameserver -> IP returned & cached
CIDR /N	N = bits for network portion. Remaining bits = host addresses. Fewer network bits = more hosts
/24 vs /16	/24: 8 host bits = 256 addrs (254 usable). /16: 16 host bits = 65,536 addrs. Each freed bit DOUBLES host count
Private ranges	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 — not publicly routable
Key ports	22 SSH · 80 HTTP · 443 HTTPS · 53 DNS · 3306 MySQL · 5432 PostgreSQL · 6379 Redis

Quiz Recap

Q1. Step by step: typing google.com to page load?

Correct (add the root->TLD->authoritative hop chain) — Cache check -> resolver queries root servers -> TLD (.com) servers -> google.com's authoritative nameservers return the IP -> IP cached & returned -> browser connects directly to that IP.

Q2. /24 meaning and usable hosts?

Correct — 24 bits network, 8 bits host = 256 addresses, 254 usable (1 reserved for network address, 1 for broadcast).

Q3. Why does /16 give MORE addresses than /24?

Corrected — needed the actual math, not just the rule — /N sets network bits; the rest are host bits. /24 leaves 8 host bits ($2^8=256$). /16 leaves 16 host bits ($2^{16}=65,536$). Every freed bit DOUBLES the host count — pure binary math, not just a memorized rule.

Q4. Ports for SSH, HTTPS, DNS — why does knowing them matter?

Corrected — reasoning was too vague — 22 (SSH), 443 (HTTPS), 53 (DNS). Turns a vague 'can't connect' into a specific test — e.g. `ss -tulnp | grep :22` to check if SSH is even listening, vs checking :443 for a website issue. Different port = different service = different fix.

Linux takeaway: The underlying knowledge is solid and retrievable — most gaps were in articulating the *why* clearly (SIGTERM vs SIGKILL, DNS-first troubleshooting order, the binary math behind CIDR) rather than not knowing the concept at all. That's the good kind of rust: it comes back fast with review, which is exactly why spaced revision like this works.

SHELL SCRIPTING

1. Variables, Arguments, User Input

Concept

<code>#!/bin/bash</code>	Shebang — must be the very first line, tells the OS which interpreter to use
<code>name="value"</code>	No spaces around = — spaces make bash think it's a command with arguments
<code>\$1, \$2, \$#</code>	Positional arguments and argument count passed to the script
<code>\$@</code> vs <code>\$*</code>	<code>"\$@"</code> keeps each argument separate (preserves spaces/quoting). <code>"\$*"</code> flattens all into one string
<code>read -p "msg" var</code>	Prompt + capture user input in one line

Quiz Recap

Q1. Why does the shebang matter?

Correct — Tells the OS which interpreter runs the script. Without it (or run with the wrong shell, e.g. sh instead of bash), bash-specific syntax can fail or behave differently.

Q2. `./deploy.sh prod v2.1` — what are `$1`, `$2`, `$#`?

Correct — `$1=prod`, `$2=v2.1`, `$#=2`

Q3. What's wrong with `name = "Alice"`?

Correct — Spaces around = make bash parse it as a command called 'name' with arguments — syntax error. Correct: `name="Alice"` (no spaces).

Q4. `$@` vs `$*` — when does the difference matter?

Learned (was unsure where it mattered) — With an argument containing spaces (e.g. "hello world"), `"$@"` in a loop preserves it as ONE item; `"$*"` flattens everything into a single string, losing that boundary. Almost always use `"$@"` when looping/passing args along.

2. if/elif/else and case Statements

Concept

<code>["\$1" == "x"]</code>	Spaces around [] are mandatory — [is a command (test), not special syntax
<code>-eq/-lt/-gt</code>	NUMERIC comparisons only
<code>== / !=</code>	STRING comparisons only — mixing these up is a classic silent bug
<code>[[]]</code>	Bash's modern, more forgiving version — supports <code>&&/ </code> , regex (<code>=~</code>)
<code>case "\$1" in ... esac</code>	Cleaner than long if/elif for one variable vs many fixed values; blocks end in <code>;;</code>

Quiz Recap

Q1. Why does `[ "$1"=="start" ]` (no spaces) fail?

Correct — [is a command, needs its arguments properly space-separated to parse; without spaces it's read as one malformed argument.

Q2. Write a case checking `$1` for yes/no/default.

Learned pattern (didn't know it initially) — `case "$1" in yes) ... ;; no) ... ;; *) ... ;; esac`

Q3. `-eq` vs `==` — which for numbers, which for strings?

Corrected — had it backwards initially — `-eq/-lt/-gt` = numbers. `==/!=` = strings. Danger: `[ "03" == "3" ]` is false as strings but `[ "03" -eq "3" ]` is true numerically — a silent trap with leading zeros (e.g. from date +%M).

Q4. Why prefer case over long if/elif for one variable, many values?

Learned (was unsure) — Avoids repeating ["\$1" == ...] every branch — states the variable once, lists bare patterns after. Shorter, harder to typo, supports pattern matching (e.g. [Yy]es)).

Practice note: extra reps on this item caught real syntax bugs — missing closing quotes, missing ;; before *) — good reminder these need hands-on typing, not just reading.

3. for, while, and until Loops

Concept

<code>for i in {1..10}; do ... done</code>	Iterate over a list/range; needs ; or newline before do
<code>while [cond]; do ... done</code>	Runs WHILE the condition is true, checked before each loop
<code>until [cond]; do ... done</code>	Runs WHILE the condition is FALSE — stops the moment it becomes true
<code>while read -r line; do ... done < file</code>	Standard safe pattern for reading a file line by line
<code>break / continue</code>	Exit the loop entirely / skip to the next iteration

Quiz Recap

Q1. Functional difference between while and until?

Corrected — had the until logic backwards — while loops WHILE the condition is true. until loops WHILE the condition is FALSE, stopping the instant it becomes true — e.g. 'retry until server responds' loops while 'responds' is false.

Q2. for loop printing 1 to 10 using {1..10}?

Correct logic, missing ; before do initially — `for i in {1..10}; do echo "Number: $i"; done`

Q3. Reading a file line by line — which loop, and what does read -r do?

Corrected — read -r explanation was wrong (not about arrays) — `while read -r line; do ... done < file.txt`. -r disables backslash-escape interpretation, so lines with backslashes (paths, trailing \) are read literally instead of being silently mangled/merged.

Q4. break vs continue?

Correct — `break` exits the loop immediately (e.g. stop checking servers once one responds). `continue` skips to the next iteration (e.g. skip empty files but keep processing the rest).

4. Functions — Arguments and Return Values

Concept

<code>name() { ... }</code>	Function definition; \$1/\$2 inside refer to args passed to the FUNCTION call
<code>return</code>	Only gives an exit status (0-255) — NOT how you get data back
<code>echo + \$(...)</code>	The real 'return a value' pattern: function echoes result, caller captures with <code>\$(...)</code>
<code>local var=...</code>	Scopes a variable to the function only — prevents silently overwriting a same-named outer variable

Quiz Recap

Q1. Why can't return give back a string?

Correct — `return` only gives an exit status (0-255) meant for success/failure signaling, not actual data.

Q2. Write a function 'add' that prints the sum of two numbers.

Corrected after 2 attempts — initially echoed inputs instead of computing/echoing the result — `add() { local sum=$(( $1 + $2 )); echo "$sum"; }`

Q3. Capture add's output into a variable 'result'?

Corrected — capture happens at the CALL SITE with \$(...), not inside the function — `result=$(add 3 5)`

Q4. What does local protect against — concrete scenario?

Correct concept, needed a concrete example — Without `local`, a function's variable can silently overwrite a same-named variable elsewhere in the script (e.g. a script-wide 'count' variable clobbered by a function's own loop)

counter) — no error, just wrong values later.

Key drilled pattern: compute the result -> echo ONLY the result -> capture with `$()` at the call site. Took 3 attempts to get right — worth re-testing in a few days.

5. Text Processing — grep, awk, sed, sort, uniq

Concept

grep -i / -r / -v / -c / -n	Case-insensitive / recursive / invert-match / count / line numbers
awk -F, '{print \$2}'	Column-based processing; -F sets the field separator (default: whitespace)
sed 's/old/new/' vs '/g'	Replace first occurrence per line vs ALL occurrences per line
sort -n / -r / -k2	Numeric / reverse / sort by column 2
uniq	Only removes ADJACENT duplicates — sort uniq is the standard combo

Quiz Recap

Q1. Why does uniq alone often fail to remove duplicates?

Correct — uniq only catches duplicates that are physically adjacent in the file — sort first so identical lines are next to each other, then uniq.

Q2. Print the 3rd column of a CSV with awk?

Corrected — forgot -F, for CSV — awk -F, '{print \$3}' file.csv — the -F, is essential; without it awk splits on whitespace and prints the whole line on a real CSV.

Q3. sed 's/fool/bar/' vs 's/fool/bar/g'?

Correct — Without g: replaces only the first match per line. With g: replaces all matches per line.

Q4. Explain: grep ERROR app.log | awk '{print \$1}' | sort | uniq -c

Correct shape, sharpened the uniq -c description — Find ERROR lines -> extract column 1 -> sort so duplicates are adjacent -> uniq -c collapses each run of identical adjacent lines into one, prefixed with its count (e.g. '14 192.168.1.5').

6. Error Handling — set -e, set -u, set -o pipefail, trap

Concept

set -e	Exit immediately on ANY command failure, instead of silently continuing
set -u	Error on undefined variables (catches typos) instead of substituting empty string
set -o pipefail	Pipeline fails if ANY stage fails, not just the last command
set -euo pipefail	The common production-script baseline, all three combined
trap 'cmd' EXIT	Runs cleanup no matter how the script ends — success, failure, or Ctrl+C

Quiz Recap

Q1. Without set -e, what happens if a command fails mid-script?

Corrected — initially described what set -e DOES, not the default (unprotected) behavior — Bash just prints an error (if any) and CONTINUES to the next line anyway — e.g. if cd fails, the next command still runs, but in the wrong directory.

Q2. Typo \$enviroment without vs with set -u?

Correct — Without -u: silently substitutes empty string, continues wrong. With -u: errors immediately with 'unbound variable', catching the typo on the spot.

Q3. Why does pipefail matter?

Correct concept, wording tightened — By default a pipeline's exit status is only the LAST command's. If an earlier stage (e.g. grep on a missing file) fails but a later stage still 'succeeds' on empty input, the whole pipeline reports success, hiding the real failure. pipefail makes the pipeline fail if ANY stage failed.

Q4. Why is trap ... EXIT more reliable than a cleanup line at the end?

Correct — A plain cleanup line never runs if the script crashes or is interrupted before reaching it. `trap ... EXIT` fires regardless of HOW the script ends — success, failure, or `Ctrl+C` — guaranteeing cleanup (temp files, locks).

7. Scheduling Scripts with crontab

Concept

min hour day month weekday	The 5 cron fields, in that order — * means "every"
0 3 * * *	Runs ONCE at 3:00 AM. Minute must be 0, not * — * in minute = runs 60x that hour!
*/15 * * * *	Step value — every 15 units in that field
Cron's minimal environment	No PATH/profile loaded — scripts that work manually can silently fail under cron
>> log 2>&1	Appends stdout AND stderr to a log file — cron shows no output by default

Quiz Recap

Q1. Crontab entry for cleanup.sh at 3:30 AM?

Correct — 30 3 * * * /home/user/cleanup.sh

Q2. */10 * * * * in plain English?

Correct — Every 10 minutes.

Q3. Script works manually but silently fails under cron — why?

Learned (was unsure) — **genuine real-world gotcha** — Cron runs with a minimal environment (no PATH/profile loaded) — a command found fine interactively may not be found under cron. Fix: use absolute paths, or set PATH explicitly at the top of the crontab.

Q4. Why add >> /var/log/job.log 2>&1 to a cron entry?

Correct — >> appends stdout to the log file. 2>&1 redirects stderr into the same place as stdout, so both normal output and errors land in one file — essential since cron jobs run silently otherwise.

Task 3 cross-check: the quick-fire quiz caught this exact bug again — wrote * 3 * * * instead of 0 3 * * * for "3 AM daily," which would fire 60 times between 3:00-3:59. This is the single most important shell/cron detail to keep drilling.

Shell Scripting takeaway: Tool mechanics were consistently solid (grep/awk/sed, loops, error flags). Real bugs clustered around syntax strictness (missing quotes/semicolons — needs hands-on typing, not just reading) and the cron minute-field trap, which is worth burning into memory since it fails silently.

GIT & GITHUB

1. Init, Stage, Commit, View History

Concept

Working Dir -> Staging -> Repo	git add moves changes to staging; git commit records them permanently
git status	Shows exactly which area your changes are in — check this whenever unsure
git log --oneline --graph	Condensed, visual history — most-used day to day
git diff vs git diff --staged	Unstaged changes (working dir vs staging) vs staged changes (staging vs last commit)

Quiz Recap

Q1. Why does Git have a separate staging step?

Correct — Lets you build a commit deliberately — e.g. 5 files edited but only 3 belong in this commit; add lets you choose exactly what's in the next snapshot.

Q2. Edited 3 files, want to commit only 2 — commands?

Correct — git add file1.txt file2.txt, then git commit -m "message" — the untouched 3rd file stays uncommitted.

Q3. git diff vs git diff --staged?

Correct — diff = unstaged changes (working dir vs staging). diff --staged = staged changes (staging vs last commit).

Q4. git log vs git log --oneline?

Correct — log = full detail per commit (message, author, date). --oneline = compact, one line per commit, best for quick scanning or with --graph.

2. Creating and Switching Branches

Concept

Branch = a pointer	Not a copy of files — just a small label pointing at a commit hash. This is why branching is cheap
git checkout existing-branch	SWITCHES to a branch that already exists — does NOT create one
git checkout -b new-branch	CREATES a new branch AND switches to it, in one step
git branch -d vs -D	-d refuses if there's unmerged work (safe). -D force-deletes regardless
main is just a convention	No special technical status in Git itself — just the default name by convention

Quiz Recap

Q1. Why is creating a branch 'cheap'?

Correct concept, mechanism needed filling in — It's just writing a new pointer (a small file with a commit hash) — no files are copied or duplicated, unlike old VCS systems where branching meant copying the whole codebase.

Q2. checkout -b vs plain checkout — which creates, which switches?

Corrected — **had these two roles swapped initially** — Plain checkout SWITCHES to an existing branch only. checkout -b CREATES a new branch AND switches to it.

Q3. git branch -d vs -D?

Correct — -d = safe delete, refuses if unmerged work would be lost. -D = force delete regardless.

Q4. Is main technically special to Git?

Correct — No — purely a naming convention. Git treats it exactly like any other branch, just a pointer that moves as commits happen.

Drilled again: confirmed with a follow-up round that `git checkout -b existing-branch` correctly errors ("branch already exists") — the create-vs-switch direction is now solid.

3. Push/Pull, Clone vs Fork

Concept

git pull = git fetch + git merge	fetch downloads without touching your files; pull fetches AND merges immediately
git clone	Git-level command — full copy of ANY repo you have access to, onto your machine
Fork	GitHub-specific feature — copy of someone else's repo under YOUR GitHub account
Contribution flow	Fork -> clone YOUR fork -> commit locally -> push to YOUR fork -> open PR back to original
origin	Just a conventional name for "the remote you cloned from" — not a Git keyword

Quiz Recap

Q1. fetch vs pull — why choose fetch on a shared branch?

Correct — fetch downloads remote changes without merging (review first, safer). pull fetches AND merges immediately — riskier on shared branches where you want to check for conflicts first.

Q2. Clone vs fork?

Correct — Clone = Git-level, downloads any repo you can access to your machine. Fork = GitHub-specific, creates your own copy under your GitHub account, used to get write access for contributing.

Q3. No write access, want to contribute a fix — full sequence?

Corrected — initially skipped the clone/push-to-fork middle steps — Fork the repo -> clone YOUR fork locally -> make changes, commit -> push to YOUR fork -> open a Pull Request back to the original. Never touch the original repo directly.

Q4. What is origin?

Correct — A conventional name, not a Git keyword — can be renamed, almost everyone leaves it as origin.

4. Merging — Fast-Forward vs Merge Commit

Concept

Fast-forward	Happens when target branch (main) hasn't moved — Git just moves the pointer forward, no new commit
Merge commit	Happens when BOTH branches have new commits — Git creates a new commit with TWO parents
Deciding factor	Has the target branch moved since the branches diverged? No -> FF. Yes -> merge commit
--no-ff	Forces a merge commit even when FF was possible — preserves visible record of the feature branch

Quiz Recap

Q1. What decides fast-forward vs merge commit?

Correct — Whether the target branch (main) has moved since the branches diverged. No movement -> fast-forward. Movement on both sides -> merge commit.

Q2. Is a new commit created in a fast-forward?

Correct — No — Git just moves the branch pointer forward to where the other branch already is.

Q3. Why does a merge commit have two parents?

Correct — Represents that development happened in parallel on two branches before being joined back together.

Q4. Why use --no-ff even when FF would work?

Correct — Preserves the exact workflow history — shows a feature branch existed and was merged, rather than flattening it into what looks like linear commits directly on main.

Clean round — all four correct on the first pass, no corrections needed.

5. Rebase — What It Does, When to Use It

Concept

Rebase	Creates NEW commits (new hashes) replaying your changes on top of the target branch — doesn't move originals
Merge vs rebase	Merge preserves branching + creates a merge commit. Rebase produces linear history, no merge commit
Golden rule	NEVER rebase commits already pushed/pulled by others — rewritten hashes cause diverged history for the team
Safe use case	Your own local, not-yet-shared branch — perfectly safe

Quiz Recap

Q1. Does rebase move or create commits?

Correct — Creates NEW commits with new hashes, replaying the same changes on top of the target branch — originals are discarded, not moved.

Q2. The golden rule of rebase, and why breaking it causes problems?

Correct — Never rebase commits already pushed/pulled by others — rewriting history gives them new hashes, so anyone who already has the old commits ends up with a diverged, conflicting history once the rewritten version is force-pushed.

Q3. Clean linear history vs accurate record of parallel branches — which fits which?

Correct — Rebase -> clean linear history. Merge -> accurate record of exactly when branches diverged/rejoined.

Q4. Safe to rebase a local, unpushed feature branch onto main?

Correct — Yes — since nobody else has it yet, rewriting its history causes no conflicts for anyone else.

Confirmed hands-on (Task 2): rebased a test commit onto main and verified its hash actually changed — because a commit's hash depends on its parent, and rebase gives it a new parent.

6. git stash and git stash pop

Concept

git stash	Temporarily shelves uncommitted changes, reverts working dir to last commit
git stash pop	Reapplies the most recent stash AND removes it from the list
git stash apply	Reapplies but KEEPS a copy in the stash list too
Stash indexing	stash@{0} = NEWEST (most recent). Higher numbers = older. Counts backward from most recent!

Quiz Recap

Q1. Real scenario for git stash?

Correct — Mid-edit on a feature branch, need to urgently switch to main to fix a bug — stash shelves the in-progress work so the branch switch is clean, then pop restores it later.

Q2. pop vs apply?

Correct — pop applies and removes the stash from the list (permanent). apply applies but keeps a copy stored (like a photocopy).

Q3. Stash twice — which index is the FIRST one made?

Corrected — initially thought **stash@{0} was the oldest/first made** — **stash@{1}**, not **stash@{0}**. Indexing counts backward from most recent: **stash@{0}** is always the NEWEST stash.

Q4. After git stash, where did the work go?

Correct — Saved into the stash (not lost) — working directory reverts to match the last commit until popped/applied back.

7. Cherry-Pick a Commit from Another Branch

Concept

git cherry-pick	Applies ONE specific commit onto your current branch — creates a copy, not the original
Real use case	Pull a critical bugfix from develop onto production without the rest of develop's unfinished work
Conflict handling	git add then git cherry-pick --continue

Quiz Recap

Q1. Real scenario for cherry-pick over merging the whole branch?

Correct — A critical bugfix sits on develop but you need ONLY that fix on production, without pulling in other half-finished work also on develop.

Q2. Move or copy — how does this relate to rebase?

Correct — Creates a copy with a new hash — same principle as rebase: originals aren't moved, new commits are created replaying the same changes elsewhere.

Q3. Cherry-pick hits a conflict — steps to resolve?

Correct — Fix the conflicting files, git add , then git cherry-pick --continue.

Q4. Grab one bugfix commit from develop onto production — two commands?

Correct — git checkout production, then git cherry-pick

Clean round — all four correct, including the cherry-pick/rebase conceptual link.

8. Squash Merge vs Regular Merge

Concept

Regular merge	main gets ALL individual commits from the feature branch, plus a merge commit
Squash merge	main gets exactly ONE commit representing the whole feature; original commits stay only on the feature branch
Why squash	Clean, readable main history — one commit per feature, easy to scan/revert
Why regular merge	Preserves granular history — useful for blame/bisect debugging on individual commits

Quiz Recap

Q1. Difference in main's resulting history?

Correct — Squash: one clean commit per feature. Regular: every individual commit from the branch stays visible, plus a merge commit.

Q2. What happens to the feature branch's original commits after squash?

Correct — They still exist on the feature branch itself — main just never sees them individually, only the single squashed commit.

Q3. Why would a team prefer squash merge for PRs?

Correct — Eliminates messy intermediate commits (WIP, typo fixes) from cluttering main's history — only the final squashed feature commit shows.

Q4. Why would a team prefer regular merge instead?

Correct — Preserving individual commits matters for pinpointing exactly which small commit introduced a bug (e.g. git bisect/blame) — a single squashed commit hides that granularity.

9. git reset (soft/mixed/hard) and git revert

Concept

reset --soft	Moves branch pointer back, keeps changes STAGED
reset --mixed (default)	Moves pointer back, keeps changes but UNSTAGED
reset --hard	Moves pointer back, DISCARDS changes completely — no undo, most dangerous
git revert	Adds a NEW commit that undoes a previous one — history stays intact, always safe on shared commits
Rule of thumb	Local/unshared commits -> reset is fine. Pushed/shared commits -> always revert, never reset

Quiz Recap

Q1. Difference between --soft, --mixed, --hard?

Correct — soft: changes go back to staged. mixed: changes unstaged but kept in working dir. hard: changes discarded entirely.

Q2. Why is reset --hard dangerous?

Correct — It permanently discards uncommitted work with no undo, and rewriting already-pushed/pulled history causes the same divergence problems as rebase on shared branches.

Q3. What does revert do differently — does it remove the original commit?

Correct (minor: it doesn't 'replace,' both stay visible side by side) — No — revert adds a new commit that cancels out the original's changes; the original stays fully visible in history alongside the revert commit.

Q4. Pushed a bad commit, teammate already pulled it — reset or revert?

Correct — Revert — reset would rewrite shared history, causing a mismatch/conflict with what the teammate already has locally.

10. GitFlow, GitHub Flow, Trunk-Based Development

Concept

GitFlow	main/develop/feature/release/hotfix branches — best for scheduled, versioned releases
GitHub Flow	Just main + short-lived feature branches -> PR -> merge. Best for frequent deploys
Trunk-Based	Direct-to-main or hours-long branches, relies on feature flags — needs strong CI/CD + testing discipline
Match strategy to deploy frequency	Scheduled releases -> GitFlow. Frequent/continuous deploys -> GitHub Flow or Trunk-Based

Quiz Recap

Q1. Why does GitFlow fit scheduled-release projects?

Correct — develop collects finished features, release/* branches let the team stabilize/test a specific version before shipping, hotfix/* patches production without disturbing in-progress work on develop.

Q2. Core difference GitHub Flow vs GitFlow?

Correct — GitHub Flow collapses GitFlow's five branch types down to just main + short-lived feature branches — no separate develop/release layer.

Q3. What does trunk-based development require to avoid chaos?

Correct — Strong CI/CD, heavy automated testing, and real engineering discipline — without it, direct-to-main commits get chaotic fast.

Q4. SaaS deploying multiple times a day — best fit, and why does GitFlow slow them down?

Corrected — initial reasoning was too vague ("slows down because of development") — GitHub Flow fits best. GitFlow slows them down because every change must pass through develop -> release -> main before reaching production — extra integration ceremony with no benefit when every merge to main IS effectively a release.

11. GitHub CLI (gh) — Repos, PRs, Issues

Concept

gh repo create / clone	GitHub-aware repo operations — shorthand owner/repo naming, uses your gh auth
gh pr create / list / checkout / merge	Manage Pull Requests fully from the terminal
gh issue create / list / close	Manage issues from the terminal
Why it matters for CI/CD	Pipelines are fully automated — gh commands can run non-interactively in scripts; a browser needs a human

Quiz Recap

Q1. git clone vs gh repo clone — meaningful difference?

Corrected — initially said 'basically the same' — gh repo clone is GitHub-aware: accepts shorthand owner/repo (no full URL needed) and uses your gh auth for private repos more seamlessly. Functionally both clone the repo, but gh is more convenient within GitHub's ecosystem.

Q2. Open a PR from current branch titled 'Fix login bug'?

Correct — `gh pr create --title "Fix login bug" --body "..."`

Q3. Review coworker's PR #57 locally, run/test the code?

Correct — `gh pr checkout 57`

Q4. Why does CI/CD specifically benefit from gh over browser workflows?

Corrected — initial answer just repeated the concept text instead of reasoning it through — CI/CD pipelines are fully automated with no human present to click through a UI — gh commands are script-callable and can run non-interactively as part of an automated job (e.g. auto-opening a PR or filing an issue on failure), which a browser-only workflow cannot do.

Git & GitHub takeaway: Strongest section overall — most core mechanics (branching, merging, rebase, reset/revert, stash) were correct on the first pass. Real slips: the checkout -b create-vs-switch direction, stash's backward indexing (0 = newest), and two late-session answers where the closest memorized phrase was reached for instead of actually reasoning it through — worth watching for that pattern near the end of a long session.

DAY 28 TASKS — WEAK SPOTS, QUICK-FIRE, TEACH-BACK

Task 2 — Weak Spots Revisited (Hands-On)

Cron minute field	* 3 * * * fires 60x (every minute of hour 3). 0 3 * * * fires once, at exactly 3:00 AM. Confirmed hands-on via crontab -l.
chown vs chmod	chown changes WHO owns a file (user/group) — never touches permission bits. chmod changes WHAT owner/group/others can do — never touches ownership. Confirmed: chmod 640 set perms, then chown root changed only the owner field in ls -l, permissions stayed 640.
Rebase copy + stash indexing	Confirmed a commit's hash changes after rebase (parent changed from A to E). Confirmed stash@{0} = newest, stash@{1} = older, by stashing twice and popping the correct index.

Task 3 — Quick-Fire Quiz Recap

Q1. chmod 755 script.sh?

Corrected — answer was incomplete — Sets ALL three categories explicitly: owner=rwx, group=r-x, others=r-x — not just 'adds execute' (that's +x).

Q2. Process vs service?

Corrected — 'starts on boot' wrongly attributed to processes in general — A process is any running program instance — can start anytime, for any reason. A service is specifically a process managed by an init system (systemd), typically persistent, controllable via systemctl.

Q3. Find what's using port 8080?

Correct — ss -tulnp | grep :8080 (or lsof -i :8080)

Q4. What does set -euo pipefail do?

Correct concept, sharpened the specifics — -e: exit on any failure. -u: error on undefined variables. -o pipefail: pipeline fails if any stage fails, not just the last. Together: fail fast and loud instead of continuing silently broken.

Q5. reset --hard vs revert?

Correct — reset --hard rewrites history, discarding commits entirely. revert preserves history, adding a new commit that undoes a previous one's changes.

Q6. Branching strategy for 5 devs shipping weekly?

Corrected — trunk-based was the wrong fit for this cadence — GitHub Flow — enough structure for PR review without GitFlow's heavy release-branch ceremony. Trunk-based needs CI/CD maturity built for multiple-times-a-day deploys, which weekly shipping doesn't require.

Q7. What does git stash do, and when to use it?

Correct — Temporarily shelves uncommitted changes so you can switch context cleanly (e.g. urgent bugfix on another branch), then restore them later.

Q8. Schedule a script to run daily at 3 AM?

Corrected — wrote * 3 * * * initially — 0 3 * * * /path/to/script.sh — minute must be 0, not *, or it fires 60 times that hour instead of once.

Q9. git fetch vs git pull?

Correct — fetch downloads remote changes without merging. pull fetches AND merges immediately.

Q10. What is LVM and why use it over regular partitions?

Correct, strong complete answer — PV (pvcreeate) -> VG (vgcreate) -> LV (lvcreate) — flexible, resizable storage. Regular partitions are fixed-size; growing them means backup/repartition/restore, while LVM resizes live in seconds if the VG has free space.

Overall Day 1-27 Revision Summary: Knowledge across Linux, Shell Scripting, and Git & GitHub is solid and largely retrievable — the consistent pattern across all three areas was strong grasp of mechanics with occasional gaps in articulating the underlying *why*, plus a handful of genuine syntax/logic flips worth re-testing in a few days: the cron minute field, checkout -b direction, stash indexing, and the set -e default-behavior question.